LifeLens

本地 AI 屏幕活动记录仪 —— 永不联网的桌面行为日志

项目说明正文 8 页（含本封面）· 代码仓库见正文首页 · 协议 MIT

LifeLens · 本地 AI 屏幕活动记录仪

数据全程不出本机。每 20 秒截一帧屏，交给本机的视觉模型看懂“我在干嘛”，再把结果沉淀成热力图、时间线、看板、日记、问答 五个视角——截屏、视觉推理、汇总、检索全部在本地完成，不调用任何云端 API、不上传任何字节。

仓库 github.com/taonai4097-prog/LifeLens · 开源协议 MIT · 界面图表 Chart.js（内置，离线可用）

> **一句话：**把“我这一天到底干了啥”变成一本随时能翻的**本地账本**——而且账本锁在你自己机器里，谁也看不到。它不替你记，只替你**不用出卖隐私也能回头看**。

一、两个真实的困境

困境一：时间都去哪了

一天结束想复盘，脑子里往往只剩“好像忙了一天”，说不清**到底干了啥、在哪个项目上花最久、被什么打断了多少次。**

想记日志，手写坚持不下来；用番茄钟只记了开始时间，中途切去刷视频、逛网页根本没记。等到周报要写“这周做了什么”，又得靠回忆硬凑。

困境二：要记录，就得先出卖隐私

市面上的录屏 / 时间追踪工具，多数要把截屏或窗口标题**传上服务器**才做分析——金融、密码、病历这类窗口一旦上传就泄露。对医院、律所、出不了内网的环境，合规直接否掉“上云”这条路。

即便在家用，把每天每一帧屏幕交给第三方，也是很多人过不去的心理坎。

那常规办法走得通吗

想到的办法	为什么走不通
手写日志 / 番茄钟	靠自律，大多数人坚持不过两周；且只记时间点，不记“在干嘛”，回到复盘仍要人猜
云端录屏（ScreenshotMonkey 等）	画面要传服务器才分析，**隐私裸奔**；出不了内网的环境直接不可用
RescueTime / ManicTime 类	只记域名 / 路径，不“看”画面；回到“我在干嘛”仍需人猜，且多为订阅 + 云端
自己写本地脚本	能离线，但要自己维护截屏 + 分析 + 存储 + 可视化，工作量等于重造一个应用

所以真正的问题不是“要不要记录”，而是**能不能在不把数据交出去的前提下、还不靠自律地自动记。**

二、我们的做法

一个 **Electron 桌面应用**：定期截屏，调用本机 **Ollama** 运行的视觉模型 `qwen3-vl:4b-instruct` 分析画面，用**简短的 5 句话**总结“用户正在做什么”，并把结果按**月份建目录、按小时建 JSON** 持久化到本地。

维度	取舍
隐私	金融 / 密码类窗口默认**整帧跳过**（连标题都不记录），可被 `privacyExclude` 扩展
算力	默认 **CPU 推理**（`num_gpu_layers: 0`）+ 全局推理互斥，把 GPU 显存让给你正在跑的其它 AI 任务
调度	**事件驱动**截屏（窗口切换 / 空闲暂停 / 停留上限），替代固定轮询——日均从 4320 帧降到约 150 帧
分析	**分层**：标题已能说明问题时跳过视觉模型，仅图像 / 视频 / 游戏类才真正调用推理
存储	月 / 小时分片 JSON + 代表帧去重 + 滚动清理，磁盘从约 1.3GB/天 压到约 50MB 稳态
问答	**RAG-lite**：模型自己规划时间范围与关键词的多轮检索（≤3 轮，无向量库）

五个视角围绕同一份本地数据长出来：**动态**（热力图 + 时间线）、**日记**（当日二阶汇总）、**看板**（多维可视化）、**问答**（检索增强）、以及贯穿其中的**十分钟结构化汇总。**

三、和别的方案有什么不一样

对比对象	它解决什么	关键差别
云端录屏	记录屏幕画面	它要**出本机**；本库**零上传**，截屏只在本地被模型看一眼
时间追踪（RescueTime 等）	记录用了哪些软件	它们只记域名 / 路径，不“看”画面；本库让视觉模型**看懂你在干嘛**并写出总结
手动日志	人自己记	靠自律、易断；本库**全自动**，开着我就在记
记忆 / RAG 类（AI 记人）	记住用户是谁、偏好什么	它们存**人**；本库存**行为事实**（某时段在干嘛），且全部本地
微调	把能力烧进权重	改权重要重训、不可逆；本库改配置即可，**秒级生效、可回滚**

一句话区别：大多数工具要么“出卖隐私换记录”，要么“只记软件不记画面”；本库做的是**在本地看懂画面、还不把任何字节送出去。**

四、工程与产品完成度

功能清单

- **十分钟结构化汇总**：每 10 分钟聚合窗口内截屏，产出项目名 / 分类 / ~1000 字描述 / 软件列表 / 可能遗漏的待办；连续活动复用相同项目名
- **“动态”标签页**：类 GitHub 热力图（每点 = 10 分钟，按活动类型着色）+ 可滚动时间线（项目名 + 描述 + 软件标签）+ 多维筛选
- **“日记”标签页**：日历选日，基于当天所有 10 分钟汇总做二阶汇总（总结 / 三件要事 / 高光 / 待办 / 优化建议 / 温馨提示）；打开自动补齐历史未生成日记
- **“看板”标签页**：天 / 周 / 月 / 年切换，活动占比饼图、软件排名、项目专注、按小时分布、日趋势折线 + 统计卡片
- **“问答”标签页**：基于本地记录的 agentic 检索增强问答，模型自规划时间范围与关键词，最多 3 轮检索
- **存储防爆盘（三层省空间，默认开）**：JPEG 质量 70 + 降分辨率 → 每 10 分钟仅留一张代表帧 → 旧截图 7 天滚动清理

模块划分

模块	职责
`main.js`	主进程：20s 截屏循环 + 10 分钟汇总调度 + 窗口 + IPC
`capture.js` / `analyzer.js`	截屏（desktopCapturer）/ 本机 Ollama 视觉分析
`storage.js` / `summary.js`	月/小时文件管理 + 原子写入 / 十分钟桶与汇总
`diary.js` / `analytics.js` / `qa.js`	日记二阶汇总 / 看板聚合 / 问答检索增强
`renderer/`	界面（内置 Chart.js，离线可用）

测试矩阵

类型	覆盖
离线冒烟（7 套）	storage / analyzer / summary / analytics / diary / qa / 配置——无需模型即可跑
端到端（4 套，需模型）	e2e 视觉分析 / e2e 日记 / e2e 问答 / e2e 十分钟汇总

五、效果数据

指标	数值
上传字节	**0**（截屏、推理、汇总、检索全本地）
磁盘占用	约 **1.3 GB/天** → 新策略稳态约 **50 MB** 封顶
日均截帧	**4320 → 约 150**（事件驱动替代固定轮询）
代表帧去重	截图数再降约 **30 倍**
旧帧保留	默认 **7 天**滚动清理，文本记录不受影响
视觉模型	本地 **4B**，单次推理数秒；全局推理互斥，不抢其它 AI 任务的 GPU

三个“降”都来自默认开启的策略：**事件驱动**砍掉空转帧、**代表帧去重**砍掉重复帧、**滚动清理**砍掉老帧。效果是“既记了，又不爆盘、还不抢显存”。

六、可优化的点，也是下一步

> 下面这几条不是免责声明，是**已经看清了的迭代方向**——每一条都对应一个具体的下一步。

现象	我们的判断与下一步
便携版开机弹 cmd / 双击无反应	`process.execPath` 指向临时解压目录；用 `PORTABLE_EXECUTABLE_DIR` 重新定位（已修）
白屏 + 内存暴涨	单实例锁竞态 → 双实例并存 → OOM；启动时先 `killOtherInstances()` 再抢锁（已修）
窗口标题中文乱码	PowerShell 默认 GBK 输出；base64 编码后再传 + 落库前 `sanitize()`（已修）
开了之后其它 AI 项目跑不动	Ollama 默认塞满显存；显式 `num_gpu_layers: 0` + 全局推理互斥（已修）
多显示器只捕主屏	当前捕获主显示屏整屏；下一步改造 `capture.js` 的源选择逻辑支持指定显示器
检索无向量库	当前关键词命中（中文不依赖分词）；下一步评估本地轻量向量库以提升到语义召回

另外仍在队列里：**分类模型轻量化**（让分层判断更准）、**移动端采集**（把手机屏幕也并进同一本账本）。

七、三步用起来

```
# 1. 准备（Node 18+ 与 Ollama，开源免费）
#    Ollama 默认地址 http://127.0.0.1:11434，应用启动会自动探测

# 2. 拉取本地视觉模型（一次即可）
ollama pull qwen3-vl:4b-instruct

# 3. 安装并启动
npm install
npm start          # 点“检测 Ollama” → 点“开始”即可
```

数据写入项目下的 `data/` 目录（便携版在 exe 同级的 `data/`），仓库 `.gitignore` 默认排除，避免误提交。

八、复现全部数字

```
# 离线冒烟（无需模型，几秒跑完）
npm run test:storage
npm run test:analyzer
npm run test:summary
npm run test:analytics
npm run test:diary
npm run test:qa

# 端到端（需本机已 pull 模型）
npm run test:e2e
npm run test:e2e-diary
npm run test:e2e-qa
npm run test:e2e-summary

# 打包为单文件便携 exe
npm run dist        # 产物 dist/LifeLens.exe（免安装，数据随 exe 走）
```

九、开源许可

- **代码**（Electron 应用 / 模块 / 脚本）：**MIT** © 2026 LifeLens contributors。自托管、自用、修改、再分发均可，无 copyleft 负担。
- **界面图表**使用 Chart.js（MIT），已内置 `renderer/vendor/` 便于离线运行。

开源前请务必自查：不要把你的 `data/`、截屏或任何个人日志提交进仓库。屏幕活动属于高度敏感数据，在公共设备上使用建议关掉 `storeScreenshots`，只保留文字总结。

十、它想长成什么样

把场景放大一点：**一个人坐在电脑前，一天里切了几十次窗口、开了十几个软件、被消息打断无数次**——这些瞬间转头就忘，却最能说明“时间花在了哪”。这件事值得有一本**锁在自己机器里的账本**。

LifeLens 想做的就一句话：**让“回顾一天”不再靠记忆，也不再靠把屏幕交给别人。**本地优先、零上传、运行时零第三方依赖——截屏只给本机的模型看一眼，看完就留在你自己硬盘上。

现在它还是**单机版**：一本属于你自己的本地时间账本。**“工具好不好用”和“数据安不安全”本不该二选一**——这本账本想证明，两者可以兼得。
